Document Number: P2029R1
Date: 2020-02-28
Audience: CWG
Reply-to: Tom Honermann <tom@honermann.net>

Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals

Introduction

This paper proposes substantial changes to [lex.phases], [lex.ccon], and [lex.string] intended to address the following core issues as well as several other more minor issues.

- Core Issue 411: Use of universal-character-name in character versus string literals
- Core Issue 1656: Encoding of numerically-escaped characters
- Core Issue 2333: Escape sequences in UTF-8 character literals

This paper follows a prior proposed resolution that only attempted to address CWG 2333. That proposed resolution was discussed on the core mailing list and at the June 24th, 2019 core issues processing teleconference. The resolution proposed in this paper attempts to address the feedback provided during those discussions.

The notes for CWG 2333 in the current active issues list (revision 100) state that discussion at the August 14th, 2017 core issues processing teleconference resulted in a determination that numeric escape sequences in UTF-8 character literals should be ill-formed. The issue has remained in drafting status since then.

SG16 discussed this issue during its October 17th, 2018 teleconference. The SG16 consensus was for a different resolution than is currently described in the active issues list. The SG16 consensus was based on the following observations:

- The current resolution in the active issues list contradicts existing practice. gcc, Clang, and Visual C++ all allow octal and hexadecimal escape sequences in UTF-8 literals.
- Octal and hexadecimal escape sequences in UTF-8 literals are useful for a number of purposes:
  - Embedding null characters: `u8"\0"`
  - Creating ill-formed code unit sequences for testing purposes.
  - Creating Modified UTF-8, CESU-8, and WTF-8 string literals. This entails two abilities:
    - Embedding `U+0000` as an overlong UTF-8 sequence: `u8"\xC0\x80"`
    - Embedding lone surrogate code points as individual UTF-8 code unit sequences. For example, encoding `U+D800` as `u8"\xED\xA0\x80"`. (Note that use of `\u` escapes specifying surrogate code points is ill-formed).
  - Compatibility with existing log/debug systems that output literals with non-printable characters represented with escapes so as to facilitate copy/paste of such output into code.

SG16 conducted the following poll:
Continue to allow hex and octal escapes that indicate code unit values, requiring only that they fit into the range of the code unit type?

SF	F	N	A	SA
8	1	0	0	0

In the polled question, "Continue" refers to existing implementation behavior; to maintain the current implementation status quo exhibited by gcc, Clang and Visual C++.

The proposed resolution reflects the SG16 consensus.

CWG 411 is addressed by specifying different behavior for character literals vs string literals for characters that are not representable by a single code unit. For example, when the execution character set is UTF-8, `'\u0153'` is conditionally-supported, has type `int` and an implementation-defined value, but `"\u0153"` is a character array of length 3 containing the UTF-8 encoding of U+0153 (LATIN SMALL LIGATURE OE) and a null character (`\xC5\x93\x00`).

CWG 1656 is addressed by clarifying that numeric escape sequences denote code unit values in the execution character set; that the values are not subject to conversion from the encoding of the source file to the execution character set.

Changes from P2029R0

- Addressed core wording review provided at the January 16th, 2020 core issues processing teleconference.
  - [lex.phases]p5: Reworded and made portions of the wording a non-normative note.
  - [lex.ccon]: Added new `character-escape-sequence` and `simple-escape-sequence-char` grammar productions to simplify wording.
  - [lex.ccon]: Added new `conditional-escape-sequence` and `conditional-escape-sequence-char` grammar productions to address previously missing grammar support for conditionally-supported implementation-defined escape sequences.
  - [lex.ccon]: Added new `conditional character literal`, `multicharacter literal`, `conditional wide character literal`, and `wide multicharacter literal` character literal kinds to simplify wording.
  - [lex.ccon]: Modified wide multicharacter literals to be conditionally supported.

    - [lex.ccon]: Modified wide character literals for which the `c-char-sequence` specifies a character that lacks representation in the execution wide-character set or that cannot be encoded in a single code unit to be conditionally supported.
    - [lex.ccon]p1: Restored original wording and added a drafting note that this paragraph will be editorially deleted and all occurrences of "character literal" replaced with references to the *`character-literal`* grammar production.
    - [lex.ccon]p2-p6: Replaced these paragraphs with a table that defines the kinds of character literals and their properties.
    - [lex.ccon]p7: Integrated wording for conditionally-supported implementation-defined escape sequences into this paragraph; this content was previously in a new paragraph.
    - [lex.ccon]p7: Removed the incorrect note about NL, HT, VT, and FF potentially lacking representation in the execution character set and the execution wide-character set. The note was added following a misreading of [lex.phases].
    - [lex.ccon]p7: Removed the note about UTF encodings being able to represent all simple-escape-sequences using a single code unit.
    - [lex.ccon]p7: Moved the contents of footnote 19 (explaining why `\?` is recognized as a simple escape sequence) into a note.
    - [lex.string]: Modified ordinary and wide string literals that specify characters that are not representable in the applicable character encoding to be conditionally supported.
    - [lex.string]p6,7,9-11: Replaced these paragraphs with a table that defines the kinds of string literals and their properties.
- Updated the implementation impact section to note Visual C++'s current handling of character literals for characters that lack representation in the execution character set.

Proposed resolution overview

The proposed wording changes are intended to resolve CWG 411, CWG 1656, and CWG 2333 by:

- Clarifying that hexadecimal and octal escape sequences:
    - are valid in `u8`, `u`, and `U` character literals. (CWG 2333)
    - specify values that need not correspond to valid code unit values for the applicable character encoding. (CWG 2333)
    - specify the execution-time value of character literals or individual code unit values of string literals; that the value expressed is not subject to conversion to the applicable execution character set. (CWG 1656)
    - specify values that may result in string literals that are ill-formed according to their applicable character encoding (CWG 2333).
- Clarifying that characters that cannot be specified in a character literal because they require more than one code unit in the applicable character encoding may be specified in string literals. (CWG 411)

Additionally, the wording updates are intended to:

- Specify behavior for non-ordinary character and string literals when a character lacks representation in the applicable character encoding. (Wording was previously missing)
- Specify that wide multicharacter literals are conditionally supported.
- Specify that ordinary and wide string literals that specify characters that are not representable in the applicable character encoding are conditionally supported.
- Specify that wide character literals that have a `c-char-sequence` that specifies a character that is not representable in the execution wide-character set or that cannot be encoded in a single code unit are conditionally supported.
- Remove non-normative wording that states that `wchar_t` is able to represent all members of the execution wide-character set as this contradicts wide spread existing practice (SG16 has an issue tracking this at https://github.com/sg16-unicode/sg16/issues/9).
- Modernize the wording with current style preferences and terminology.

The concept of an "associated character encoding" is introduced for character and string literals so as to enable wording to be independent of the particular kind of literal (ordinary, wide or Unicode).

New `basic-c-char`, `basic-s-char`, `character-escape-sequence`, `numeric-escape-sequence`, and `simple-escape-sequence-char` grammar productions are proposed in order to simplify wording. The `c-char`, `s-char`, and `escape-sequence` grammar productions are updated to define them in terms of the new grammar productions.

New `conditional-escape-sequence` and `conditional-escape-sequence-char` grammar production are proposed to address previously missing grammar support for conditionally-supported implementation-defined escape sequences.

New `multicharacter literal` and `wide multicharacter literal` character literal kinds are defined in order to simplify wording. These new kinds cover the cases where an ordinary or wide character literal has a `c-char-sequence` containing more than one `c-char`.

New `conditional character literal` and `conditional wide character literal` character literal kinds are defined in order to simplify wording. These new kinds cover the cases where the character specified by the `c-char-sequence` lacks representation in the execution (wide-)character set or cannot be encoded in a single code unit.

Implementation impact

The author intends the proposed wording to reflect existing practice for the gcc and Clang compilers. If this proposal is adopted, neither of those compilers are expected to require updates. However, there is implementation impact to the Microsoft Visual C++ compiler.

Semantics of numeric-escape-sequences in UTF-8 literals

Consider the following (C++20) code:

```
constexpr const char8_t c[] = u8"\xc3\x80"; // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
#if defined(_MSC_VER)
  // Microsoft Visual C++:
  static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C3 {LATIN CAPITAL LETTER A WITH TILDE}
  static_assert(c[1] == 0x83);
  static_assert(c[2] == 0xC2); // UTF-8 encoding of U+0080 {<control>}
  static_assert(c[3] == 0x80);
  static_assert(c[4] == 0x00); // null
#else
  // Gcc and Clang:
  static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
  static_assert(c[1] == 0x80);
  static_assert(c[2] == 0x00); // null
#endif
```

Gcc and Clang encode the hexadecimal escapes as code units in the target (UTF-8) encoding and perform no conversions (consistent with the behavior intended by this proposal). However, Visual C++ considers each hexadecimal escape to specify a code point in the source encoding and encodes each as UTF-8.

The author does not know if the Visual C++ behavior exhibited for UTF-8 literals is intentional or reflective of a defect. The behavior is inconsistent for UTF-8 and UTF-16 literals. For UTF-8 literals, numeric escape sequences that specify values outside the range of `char8_t` are accepted as code point values and encoded as UTF-8. However, for UTF-16 literals, numeric escape sequences that specify values outside the range of `char16_t` are rejected rather than being considered a code point value and encoded as a UTF-16 surrogate pair.

Character literal type for characters not representable in the execution character set

Consider the following code assuming that the execution character set does not have representation for the specified Unicode code points.

```
auto c1 = '\uFF10';
extern char c1;
#ifdef _MSC_VER
static_assert('\uFF10' == '?');
#endif
auto c2 = '\U0001F235';
extern int c2;
#ifdef _MSC_VER
static_assert('\U0001F235' == '??');
#endif
```

This code should be rejected (both before and after this proposal) because the redeclaration of `c1` with type `char` does not match the first declaration for which `c1` should have a deduced type of `int`. Visual C++ accepts it when compiled with `/execution-charset:windows-1252` with the following warnings:

```
<source>(1): warning C4566: character represented by universal-character-name '\uFF10' cannot be represented in the current code page (1252)
<source>(4): warning C4566: character represented by universal-character-name '\uFF10' cannot be represented in the current code page (1252)
<source>(6): warning C4566: character represented by universal-character-name '\U0001F235' cannot be represented in the current code page (1
<source>(9): warning C4566: character represented by universal-character-name '\U0001F235' cannot be represented in the current code page (1
```

It seems that the Visual C++ compiler translates unrepresentable characters from the Unicode BMP to a single `char` with value equal to `'?'`, but translates unrepresentable characters from outside the Unicode BMP to `int` with value equal to the multicharacter literal `'??'`. This seems unlikely to be intended behavior. It would be conforming if, for the Unicode BMP case, an `int` with value equal to `'?'` was produced.

Gcc and Clang both reject the above code regardless of whether those Unicode characters have representation in the execution character set. If they are representable, then the code is rejected (as permitted) because the characters cannot be encoded in a single code unit. If they are not representable (which only happens for gcc since Clang always targets UTF-8), then the code is rejected because the redeclaration of `c1` as `char` does not match the deduced `int` type for its first declaration.

Acknowledgements

Thank you to Jens Maurer and Steve Downey for providing feedback on initial drafts of this paper!

Proposed resolution

These changes are relative to N4835.

The changes to [lex.ccon] and [lex.string] are rather pervasive. For ease of review, unchanged paragraphs in these sections are retained in the wording below. These paragraphs are introduced with "*No* changes to ..." and are highlighted with a blue background.

These changes do not reflect recent editorial changes made in https://github.com/cplusplus/draft/pull/2768; merge conflict resolution will be required.

☐ Hide inserted text
☐ Hide deleted text

[lex.phases]

Change in 5.2 [lex.phases] paragraph 5:
Drafting Note: The change of "escape sequence" to "`character-escape-sequence`" addresses CWG 1656.

> Each ~~basic source character set member~~*basic-c-char*, *basic-s-char*, and *r-char* in a character literal or a string literal, as well as each ~~escape sequence~~*character-escape-sequence* and *universal-character-name* in a character literal or a non-raw string literal, is converted to the ~~corresponding member of the execution character set ([lex.ccon], [lex.string]);~~ literal's associated character encoding as specified in [lex.ccon] and [lex.string]. ~~if there is no corresponding member, it is converted to an implementation-defined member other than the null (wide) character~~ [*Note:* If a character lacks representation in the associated character encoding, then the character is converted to an implementation-defined character or the literal is ill-formed ([lex.ccon], [lex.string]) — *end note*]. [8]

Change in 5.2 [lex.phases] paragraph 7:
Drafting note: This addition duplicates wording in [lex.string], but seems important to include here.

> A null character is appended to every *string-literal*. White-space characters separating tokens are no longer significant. Each preprocessing token is converted into a token ([lex.token]). The resulting tokens are syntactically and semantically analyzed and translated as a translation unit. [*Note:* The process of analyzing and translating the tokens may occasionally result in one token being replaced by a sequence of other tokens ([temp.names]). — *end note*] It is implementation-defined whether the sources for module units and header units on which the current translation unit has an interface dependency ([module.unit], [module.import]) are required to be available. [*Note:* Source files, translation units and translated translation units need not necessarily be stored as files, nor need there be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation. — *end note*]

[lex.ccon]

Change in 5.13.3 [lex.ccon]:

character-literal:
encoding-prefix$_{opt}$ ' c-char-sequence '

encoding-prefix: one of
`u8 u U L`

c-char-sequence:
c-char
c-char-sequence c-char

c-char:
~~any member of the basic source character set except the single-quote ', backslash \, or new-line character~~
basic-c-char
escape-sequence
universal-character-name

basic-c-char:
any member of the basic source character set except the single-quote ', backslash \, or new-line character

escape-sequence:
~~simple-escape-sequence~~
~~octal-escape-sequence~~
~~hexadecimal-escape-sequence~~
character-escape-sequence
numeric-escape-sequence

character-escape-sequence:
simple-escape-sequence
conditional-escape-sequence

simple-escape-sequence: ~~one of~~
~~`\' \" \? \\`~~
~~`\a \b \f \n \r \t \v`~~
\ simple-escape-sequence-char

simple-escape-sequence-char: one of
`' " ? \`
`a b f n r t v`

conditional-escape-sequence:
\ conditional-escape-sequence-char

conditional-escape-sequence-char:
any member of the basic source character set that is not in simple-escape-sequence-char

numeric-escape-sequence:
octal-escape-sequence
hexadecimal-escape-sequence

octal-escape-sequence:
\ octal-digit
\ octal-digit octal-digit
\ octal-digit octal-digit octal-digit

hexadecimal-escape-sequence:
\x hexadecimal-digit
hexadecimal-escape-sequence hexadecimal-digit

No changes to 5.13.3 [lex.ccon] paragraph 1:
Drafting Note: Per discussion in the January 16th, 2020 core issues processing teleconference, this paragraph will be editorially deleted and all occurrences of "character literal" will be replaced with references to the *`character-literal`* grammar production.

A character literal is one or more characters enclosed in single quotes, as in `'x'`, optionally preceded by `u8`, `u`, `U`, or `L`, as in `u8'w'`, `u'x'`, `U'y'`, or `L'z'`, respectively.

Add a new paragraph and table (X) after 5.13.3 [lex.ccon] paragraph 1:

Table X specifies the kinds of *character-literal*s and their properties. The type of a *character-literal* is the associated code unit type. *conditional character literal*s and *conditional wide character literal*s are distinguished from *ordinary character literal*s and *wide character literal*s respectively by the presence of a *c-char-sequence* that contains a single *c-char* that is not a *numeric-escape-sequence* and that specifies a character that either lacks representation in the applicable associated character encoding or that cannot be encoded in a single code unit. *multicharacter literal*s and *wide multicharacter literal*s are distinguished from *ordinary character literal*s and *wide character literal*s respectively by the presence of a *c-char-sequence* that contains more than one *c-char*; the *c-char-sequence* of all other character literals contains exactly one *c-char*. *conditional character literal*s, *multicharacter literal*s, *conditional wide character literal*s, and *wide multicharacter literal*s are conditionally supported.

Table X: Character literals [tab:lex.ccon.literals]

Kind	Encoding prefix	Associated code unit type	Associated character encoding	Example
ordinary character literal	none	`char`	encoding of the execution character set	`'v'`
conditional character literal	none	`int`	implementation-defined	`'\U0001F525'` (assuming lack of representation in the execution character set, or that representation would require more than one code unit.)
multicharacter literal	none	`int`	implementation-defined	`'abcd'`
wide character literal	`L`	`wchar_t`	encoding of the execution wide-character set	`L'w'`
conditional wide character literal	`L`	`wchar_t`	implementation-defined	`L'\U0001F32A'` (assuming lack of representation in the execution wide-character set, or that representation would require more than one code unit.)
wide multicharacter literal	`L`	`wchar_t`	implementation-defined	`L'abcd'`
UTF-8 character literal	`u8`	`char8_t`	UTF-8	`u8'x'`
UTF-16 character literal	`u`	`char16_t`	UTF-16	`u'y'`
UTF-32 character literal	`U`	`char32_t`	UTF-32	`U'z'`

Delete 5.13.3 [lex.ccon] paragraph 2:
Drafting Note: The contents of paragraphs 2-6 were incorporated into new paragraphs.

~~A character literal that does not begin with `u8`, `u`, `U`, or `L` is an *ordinary character literal*. An ordinary character literal that contains a single *c-char* representable in the execution character set has type `char`, with value equal to the numerical value of the encoding of the *c-char* in the execution character set. An ordinary character literal that contains more than one *c-char* is a *multicharacter literal*. A multicharacter literal, or an ordinary character literal containing a single *c-char* not representable in the execution character set, is conditionally-supported, has type `int`, and has an implementation-defined value.~~

Delete 5.13.3 [lex.ccon] paragraph 3:
Drafting Note: The contents of paragraphs 2-6 were incorporated into new paragraphs. The note regarding the range of single code unit values was removed.

~~A character literal that begins with `u8`, such as `u8'w'`, is a character literal of type `char8_t`, known as a *UTF-8 character literal*. The value of a UTF-8 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-8 code unit. [*Note:* That is, provided the code point value is in the range 0x0-0x7F (inclusive). — *end note*] If the value is not representable with a single UTF-8 code unit, the program is ill-formed. A UTF-8 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 4:
Drafting Note: The contents of paragraphs 2-6 were incorporated into new paragraphs. The note regarding the range of singled code unit values was removed.

~~A character literal that begins with the letter `u`, such as `u'x'`, is a character literal of type `char16_t`, known as a *UTF-16 character literal*. The value of a UTF-16 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value is representable with a single 16-bit code unit. [*Note:* That is, provided the code point value is in the range 0x0-0xFFFF (inclusive). — *end note*] If the value is not representable with a single 16-bit code unit, the program is ill-formed. A UTF-16 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 5:
Drafting Note: The contents of paragraphs 2-6 were incorporated into new paragraphs.

~~A character literal that begins with `U`, such as `U'y'`, is a character literal of type `char32_t`, known as a *UTF-32 character literal*. The value of a UTF-32 character literal containing a single *c-char* is equal to its ISO/IEC 10646 code point value. A UTF-32 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 6:
Drafting Note: The contents of paragraphs 2-6 were incorporated into new paragraphs. The note regarding the ability for `wchar_t` to store all values of the execution wide-character set is intentionally removed as it conflicts with long standing existing practice (https://github.com/sg16-unicode/sg16/issues/9). The reference to footnote 18 and the footnote itself are also removed.

~~A character literal that begins with the letter `L`, such as `L'z'`, is a *wide-character literal*. A wide-character literal has type `wchar_t`.[18] The value of a wide-character literal containing a single *c-char* has value equal to the numerical value of the encoding of the *c-char* in the execution wide-character set, unless the *c-char* has no representation in the execution wide-character set, in which case the value is implementation-defined. [*Note:* The type `wchar_t` is able to represent all members of the execution wide-character set (see [basic.fundamental]). — *end note*] The value of a wide-character literal containing multiple *c-char*s is implementation-defined.~~

Delete footnote 18
Drafting Note: The reference to this footnote was removed and the note itself found to have too little value to retain elsewhere.

~~18) They are intended for character sets where a character does not fit into a single byte.~~

Add a new paragraph (X):

The value of a character literal is as follows.

(X.1) — The value of a *conditional character literal*, *multicharacter literal*, *conditional wide character literal*, or *wide multicharacter literal* is implementation-defined.

(X.2) — The value of a character literal consisting of a single *basic-c-char*, *character-escape-sequence*, or *universal-character-name* is the code unit value of the specified character encoded with the character literal's associated character encoding. If the character cannot be encoded in a single code unit, then the character literal is either a *conditional character literal*, a *conditional wide character literal*, or is ill-formed.

(X.3) — The value of a character literal consisting of a single *numeric-escape-sequence* is the numeric value of the octal or hexadecimal number. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. If the numeric value exceeds the range of the character literal's associated code unit type, then, for an *ordinary character literal* or a *wide character literal*, the value is implementation-defined and, for all other character literals, the character literal is ill-formed.

Change in 5.13.3 [lex.ccon] paragraph 7:
Drafting Note: The added note is content moved from footnote 19. The deleted text has been removed as redundant since it repeats information implicit in the grammar.

The character specified by a *simple-escape-sequence* is specified in table 8. [*Note:* Using an escape sequence for a question mark is supported for compatibility with ISO C++ 2014 and ISO C. — *end note*] *conditional-escape-sequence*s are conditionally-supported and specify an implementation-defined character. ~~Certain non-graphic characters, the single quote `'`, the double quote `"`, the question mark `?`,[19] and the backslash `\`, can be represented according to Table 8. The double quote `"` and the question mark `?`, can be represented as themselves or by the escape sequences `\"` and `\?` respectively, but the single quote `'` and the backslash `\` shall be represented by the escape sequences `\'` and `\\` respectively. Escape sequences in which the character following the backslash is not listed in Table 8 are conditionally-supported, with implementation-defined semantics. An escape sequence specifies a single character.~~

Table 8: ~~E~~Simple escape sequences [tab:lex.ccon.esc]

new-line	NL(LF)	`\n`
horizontal tab	HT	`\t`
vertical tab	VT	`\v`
backspace	BS	`\b`
carriage return	CR	`\r`
form feed	FF	`\f`
alert	BEL	`\a`
backslash	\	`\\`
question mark	?	`\?`
single quote	'	`\'`
double quote	"	`\"`
~~octal number~~	~~*ooo*~~	~~`\`*ooo*~~
~~hex number~~	~~*hhh*~~	~~`\x`*hhh*~~

Delete footnote 19 referenced from 5.13.3 [lex.ccon] paragraph 7:
Drafting Note: The footnote text was moved into a note in [lex.ccon] paragraph 7.

~~19) Using an escape sequence for a question mark is supported for compatibility with ISO C++ 2014 and ISO C.~~

Delete 5.13.3 [lex.ccon] paragraph 8:
Drafting Note: Wording describing the form of octal and hexadecimal escape sequences has been removed as redundant; the form is implicit in the grammar.

~~The escape `\`*ooo* consists of the backslash followed by one, two, or three octal digits that are taken to specify the value of the desired character. The escape `\x`*hhh* consists of the backslash followed by `x` followed by one or more hexadecimal digits that are taken to specify the value of the desired character. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. The value of a character literal is implementation-defined if it falls outside of the implementation-defined range defined for `char` (for character literals with no prefix) or `wchar_t` (for character literals prefixed by `L`). [*Note:* If the value of a character literal prefixed by `u`, `U8`, or `U` is outside the range defined for its type, the program is ill-formed. — *end note*]~~

Delete 5.13.3 [lex.ccon] paragraph 9:
Drafting Note: The normative text was combined with wording for `basic-c-char` and `character-escape-sequence` above. The deleted note duplicates normative text in 5.2 [lex.phases] paragraph 1.

~~A *universal-character-name* is translated to the encoding, in the appropriate execution character set, of the character named. If there is no such encoding, the *universal-character-name* is translated to an implementation-defined encoding. [*Note:* In translation phase 1, a *universal-character-name* is introduced whenever an actual extended character is encountered in the source text. Therefore, all extended characters are described in terms of *universal-character-name*s However, the actual compiler implementation may use its own native character set, so long as the same results are obtained. — *end note*]~~

[lex.string]

Change in 5.13.5 [lex.string]:

string-literal:
  encoding-prefix$_{opt}$ " s-char-sequence$_{opt}$ "
  encoding-prefix$_{opt}$ `R` raw-string

s-char-sequence:
  s-char
  s-char-sequence s-char

s-char:
  ~~any member of the basic source character set except the double-quote `"`, backslash `\`, or new-line character~~
  basic-s-char

escape-sequence
universal-character-name

basic-s-char:
any member of the basic source character set except the double-quote ", backslash \, or new-line character

raw-string:
" d-char-sequence$_{opt}$ (r-char-sequence$_{opt}$) d-char-sequence$_{opt}$ "

r-char-sequence:
r-char
r-char-sequence r-char

r-char:
any member of the source character set, except a right parenthesis) followed by
the initial d-char-sequence (which may be empty) followed by a double quote ".

d-char-sequence:
d-char
d-char-sequence d-char

d-char:
any member of the source character set except:
space, the left parenthesis (, the right parenthesis), the backslash \, and the control characters
representing horizontal tab, vertical tab, form feed, and newline.

Delete 5.13.5 [lex.string] paragraph 1:
Drafting Note: The contents of paragraph 1 was incorporated into new paragraphs.

~~A *string-literal* is a sequence of characters (as defined in [lex.ccon]) surrounded by double quotes, optionally prefixed by `R`, `u8`, `u8R`, `u`, `uR`, `U`, `UR`, `L`, or `LR`, as in `"..."`, `R"(...)"`, `u8"..."`, `u8R"**(...)**"`, `u"..."`, `uR"*~(...)*~"`, `U"..."`, `UR"zzz(...)zzz"`, `L"..."`, or `LR"(...)"`, respectively.~~

Add a new paragraph and table (X) after 5.13.5 [lex.string] paragraph 1:

Table X specifies the kinds of *string-literal*s and their properties.

Table X: String literals [tab:lex.string.literals]

Kind	Encoding prefix	Type	Associated character encoding	Examples

ordinary string literal	none	array of *n* `const char`	encoding of the execution character set	`""`
				`"an ordinary string"`
				`R"(an ordinary raw string)"`

wide string literal	`L`	array of *n* `const wchar_t`	encoding of the execution wide-character set	`L""`
				`L"a wide string"`
				`LR"w(a wide raw string)w"`

UTF-8 string literal	`u8`	array of *n* `const char8_t`	UTF-8	`u8""`
				`u8"a UTF-8 string"`
				`u8R"x(a UTF-8 raw string)x"`

UTF-16 string literal	`u`	array of *n* `const char16_t`	UTF-16	`u""`
				`u"a UTF-16 string"`
				`uR"y(a UTF-16 raw string)y"`

UTF-32 string literal	`U`	array of *n* `const char32_t`	UTF-32	`U""`
				`U"A UTF-32 string"`
				`UR"z(a UTF-32 raw string)z"`

No changes to 5.13.5 [lex.string] paragraph 2:

A *string-literal* that has an `R` in the prefix is a *raw string literal*. The *d-char-sequence* serves as a delimiter. The terminating *d-char-sequence* of a *raw-string* is the same sequence of characters as the initial *d-char-sequence*. A *d-char-sequence* shall consist of at most 16 characters.

No changes to 5.13.5 [lex.string] paragraph 3:

[*Note:* The characters `'('` and `')'` are permitted in a *raw-string*. Thus, `R"delimiter((a|b))delimiter"` is equivalent to `"(a|b)"`. — *end note*]

No changes to 5.13.5 [lex.string] paragraph 4:

[*Note:* A source-file new-line in a raw string literal results in a new-line in the resulting execution string literal. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

```
const char* p = R"(a\
b
c)";
assert(std::strcmp(p, "a\\\nb\nc") == 0);
```

— *end note*]

No changes to 5.13.5 [lex.string] paragraph 5:

[*Example:* The raw string

```
R"a(
)\
a"
)a"
```

is equivalent to `"\n)\\\na\"\n"`. The raw string

```
R"(x = "\"y\"")"
```

is equivalent to `"x = \"\\\"y\\\"\""`. — *end example*]

Delete 5.13.5 [lex.string] paragraph 6:
Drafting Note: The contents of paragraphs 6, 7, 9, 10, and 11 were incorporated into new paragraphs. The wording regarding translation phase 6 has been removed as inaccurate; string contents are not fully known until after phase 7.

~~After translation phase 6, a *string-literal* that does not begin with an *encoding-prefix* is an *ordinary string literal*. An ordinary string literal has type "array of *n* const char" where *n* is the size of the string as defined below, has static storage duration ([basic.stc]), and is initialized with the given characters.~~

Delete 5.13.5 [lex.string] paragraph 7:
Drafting Note: The contents of paragraphs 6, 7, 9, 10, and 11 were incorporated into new paragraphs.

~~A *string-literal* that begins with `u8`, such as `u8"asdf"`, is a *UTF-8 string literal*. A UTF-8 string literal has type "array of *n* const char8_t", where *n* is the size of the string as defined below; each successive element of the object representation ([basic.types]) has the value of the corresponding code unit of the UTF-8 encoding of the string.~~

No changes to 5.13.5 [lex.string] paragraph 8:

Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals.

Delete 5.13.5 [lex.string] paragraph 9:
Drafting Note: The contents of paragraphs 6, 7, 9, 10, and 11 were incorporated into new paragraphs. The note has been deleted as redundant; the use of surrogate pairs is explicit in the UTF-16 encoding.

~~A *string-literal* that begins with `u`, such as `u"asdf"`, is a *UTF-16 string literal*. A UTF-16 string literal has type "array of *n* const char16_t", where *n* is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-16 encoding of the string. [*Note:* A single *c-char* may produce more than one char16_t character in the form of surrogate pairs. A surrogate pair is a representation for a single code point as a sequence of two 16-bit code units. — *end note*]~~

Delete 5.13.5 [lex.string] paragraph 10:
Drafting Note: The contents of paragraphs 6, 7, 9, 10, and 11 were incorporated into new paragraphs.

~~A *string-literal* that begins with `U`, such as `U"asdf"`, is a *UTF-32 string literal*. A UTF-32 string literal has type "array of *n* const char32_t", where *n* is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-32 encoding of the string.~~

Delete 5.13.5 [lex.string] paragraph 11:
Drafting Note: The contents of paragraphs 6, 7, 9, 10, and 11 were incorporated into new paragraphs.

~~A *string-literal* that begins with `L`, such as `L"asdf"`, is a *wide string literal* as its associated character encoding. A wide string literal has type "array of *n* const wchar_t", where *n* is the size of the string as defined below; it is initialized with the given characters.~~

Change in 5.13.5 [lex.string] paragraph 12:

In translation phase 6 ([lex.phases]), adjacent *string-literal*s are concatenated. If both *string-literal*s have the same *encoding-prefix*, the resulting concatenated string literal has that *encoding-prefix*. If one *string-literal* has no *encoding-prefix*, it is treated as a *string-literal* of the same *encoding-prefix* as the other operand. If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed. Any other concatenations are conditionally-supported with implementation-defined behavior. [*Note:* This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (~~after each character from a string literal has been translated into a value from the appropriate character set~~after the string literal contents have been encoded in the *string-literal*'s associated character encoding), a *string-literal*'s initial rawness has no effect on the interpretation or well-formedness of the concatenation. — *end note*] Table 9 has some examples of valid concatenations.

Table 9: String literal concatenations [tab:lex.string.concat]

Source	Means	Source	Means	Source	Means
`u"a" u"b"`	`u"ab"`	`U"a" U"b"`	`U"ab"`	`L"a" L"b"`	`L"ab"`
`u"a" "b"`	`u"ab"`	`U"a" "b"`	`U"ab"`	`L"a" "b"`	`L"ab"`
`"a" u"b"`	`u"ab"`	`"a" U"b"`	`U"ab"`	`"a" L"b"`	`L"ab"`

Characters in concatenated strings are kept distinct.

[*Example:*

```
"\xA" "B"
```

contains the two characters `'\xA'` and `'B'` after concatenation (and not the single hexadecimal character `'\xAB'`). — *end example*]

Change in 5.13.5 [lex.string] paragraph 13:

After any necessary concatenation, in translation phase 7 ([lex.phases]), ~~`'\0'`~~a null character is appended to every string literal so that programs that scan a string can find its end.

Delete 5.13.5 [lex.string] paragraph 14:
Drafting note: This wording has been removed as misleading, incomplete, or redundant. String literal contents do not always have the same meaning as in character

literals. The wording regarding single and double quotes is redundant with the grammar. The discussion of string length is unnecessary as string length is determined by encoding.

~~Escape sequences and *universal-character-name*s in non-raw string literals have the same meaning as in character literals ([lex.ccon]), except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a *universal-character-name* in a UTF-16 string literal may yield a surrogate pair. In a narrow string literal, a *universal-character-name* may map to more than one `char` or `char8_t` element due to *multibyte encoding*. The size of a `char32_t` or wide string literal is the total number of escape sequences, *universal-character-name*s, and other characters, plus one for the terminating `U'\0'` or `L'\0'`. The size of a UTF-16 string literal is the total number of escape sequences, *universal-character-name*s, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] Within `char32_t` and `char16_t` string literals, any *universal-character-name*s shall be within the range `0x0` to `0x10FFFF`. The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each *universal-character-name*, plus one for the terminating `'\0'`.~~

Change in 5.13.5 [lex.string] paragraph 15:
Drafting note: Wording for string literal object initialization has been moved to a new paragraph.

Evaluating a *string-literal* results in a string literal object with static storage duration ([basic.stc])~~, initialized from the given characters as specified above~~. Whether all string literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a *string-literal* yield the same or a different object is unspecified. [*Note:* The effect of attempting to modify a string literal is undefined. — *end note*]

Add a new paragraph (X) after 5.13.5 [lex.string] paragraph 15:

String literal objects are initialized with the sequence of code unit values corresponding to the *string-literal*'s sequence of *s-char*s (for a non-raw string literal) and *r-char*s (for a raw string literal) after translation phase 7 ([lex.phases]) as follows.

(X.1) — The characters denoted by each *basic-s-char*, *r-char*, *character-escape-sequence* ([lex.ccon]), and *universal-character-name* ([lex.charset]) are encoded to a code unit sequence using the *string-literal*'s associated character encoding. If a character lacks representation in the associated character encoding, then, for an *ordinary string literal* or a *wide string literal*, the *string-literal* is conditionally-supported and an implementation defined code unit sequence is encoded; otherwise, the *string-literal* is ill-formed.

(X.2) — Each *numeric-escape-sequence* ([lex.ccon]) contributes a single code unit value with the numeric value of the octal or hexadecimal number. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. If the numeric value exceeds the range of the *string-literal*'s code unit type, then, for an *ordinary string literal* or a *wide string literal*, an implementation-defined code unit value is encoded; otherwise, the *string-literal* is ill-formed.